Capture the Flag 5: Make the System Interactive

Goal: Transform your Student Directory into an interactive application where administrators can favorite students, edit information, and remove records using Flutter interactions and state management.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

🏴 Flag 0 — Create Your Feature Branch

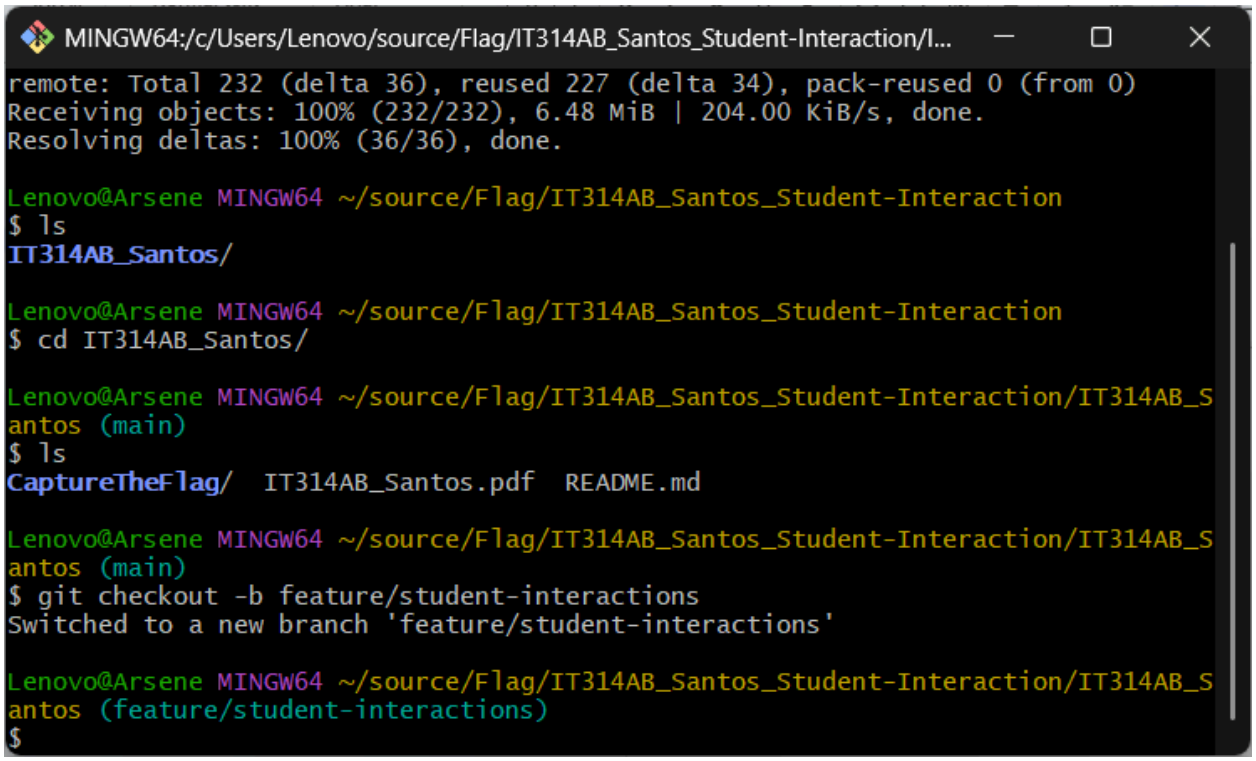
Objective: Create a new feature branch before adding interactions.

Create:

`feature/student-interactions`

Screenshot:

- Terminal showing your current branch



🏴 Flag 1 — Wake Up the Buttons

Add at least two buttons inside each Student Card.

Example actions:

- Favorite
- Edit

Objective: Make your Student Card respond to button presses using `onPressed`.

Right now your buttons are only decorations.

Research how Flutter's `onPressed` works.

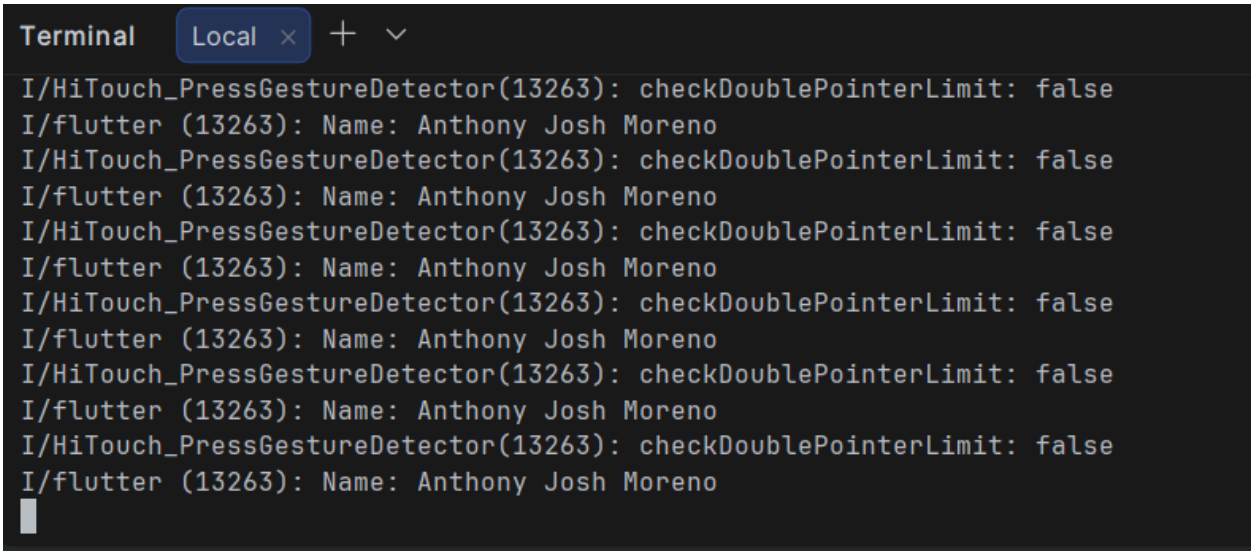
The buttons do not need to perform their final behavior yet. They simply need to respond when pressed.

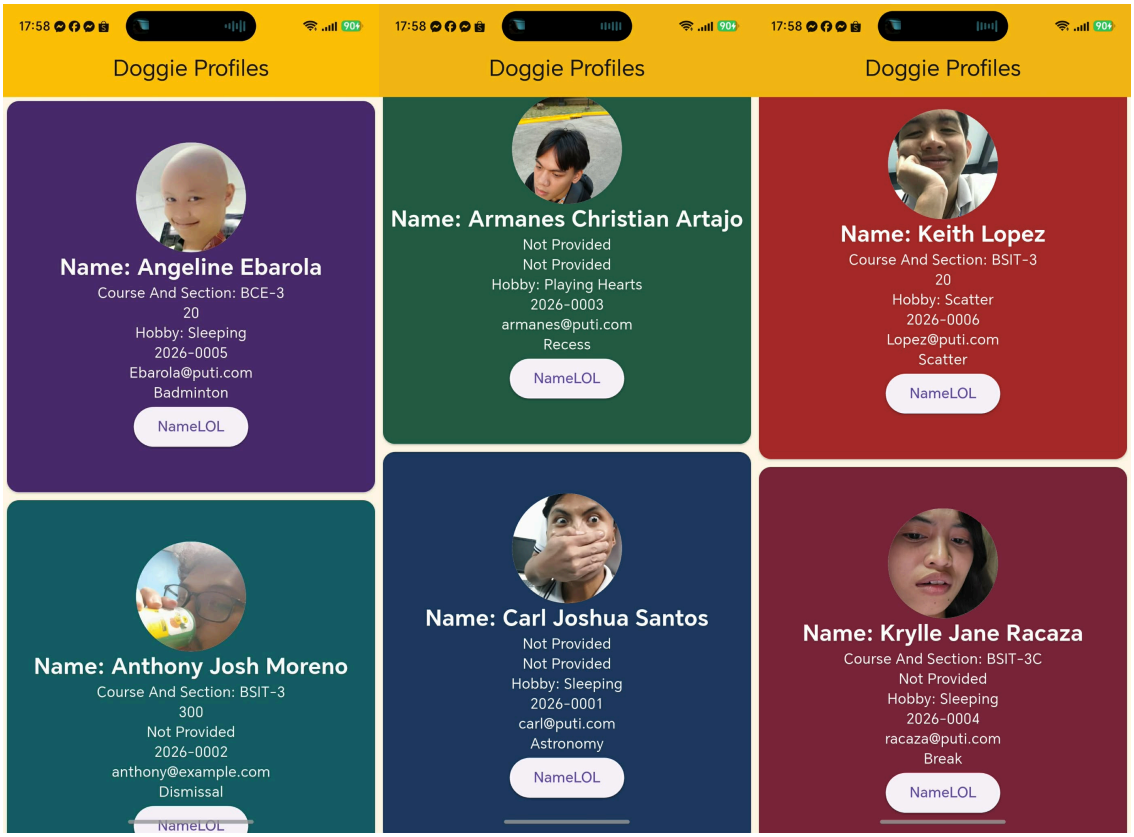
Test

When a button is pressed, your app should visibly react in some way (such as printing a message or showing a temporary notification).

Screenshot:

- Student Card with buttons
- `onPressed` inside your code
- Button reacting or terminal message







🏴 Flag 2 — Touch Anywhere

Objective: Make the entire Student Card tappable using `onTap`.

Buttons aren't the only interactive elements.

Research Flutter's `onTap`.

Wrap your Student Card so tapping anywhere on the card triggers an interaction.

Test

The user should be able to:

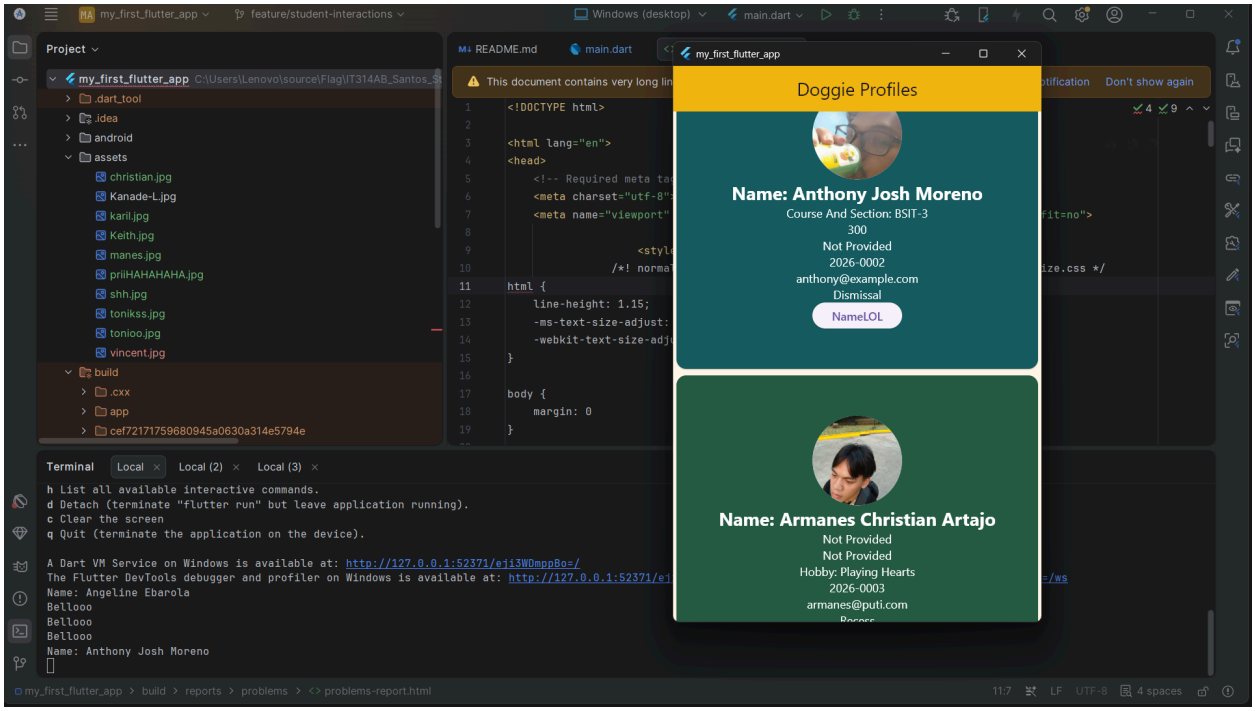
- Tap the Favorite button
- Tap the Edit button
- Tap anywhere else on the Student Card

Each interaction should be recognized separately. (One function for the 2 buttons, another for the Student Card)

Screenshot:

- `onTap` in your code
- Student Card being tapped

```
child: GestureDetector(
  onTap: ()
  {
    print("Bellooo");    Don't invoke 'print' in production
  },
  child: Card(
    color: Color(profile.color),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      crossAxisAlignment: CrossAxisAlignment.center,
      children: [
```



🏴 Flag 3 — Discover `setState`

Objective: Update the screen immediately using `setState`.

So far your app reacts. But does the UI actually change?

Research Flutter's `setState`.

Use it so pressing a button immediately updates something visible on the screen.

Examples include:

- Changing text
- Changing an icon
- Updating a counter

The important part is understanding that `setState` tells Flutter to rebuild the UI.

Test

The screen should visibly change immediately after pressing the button.

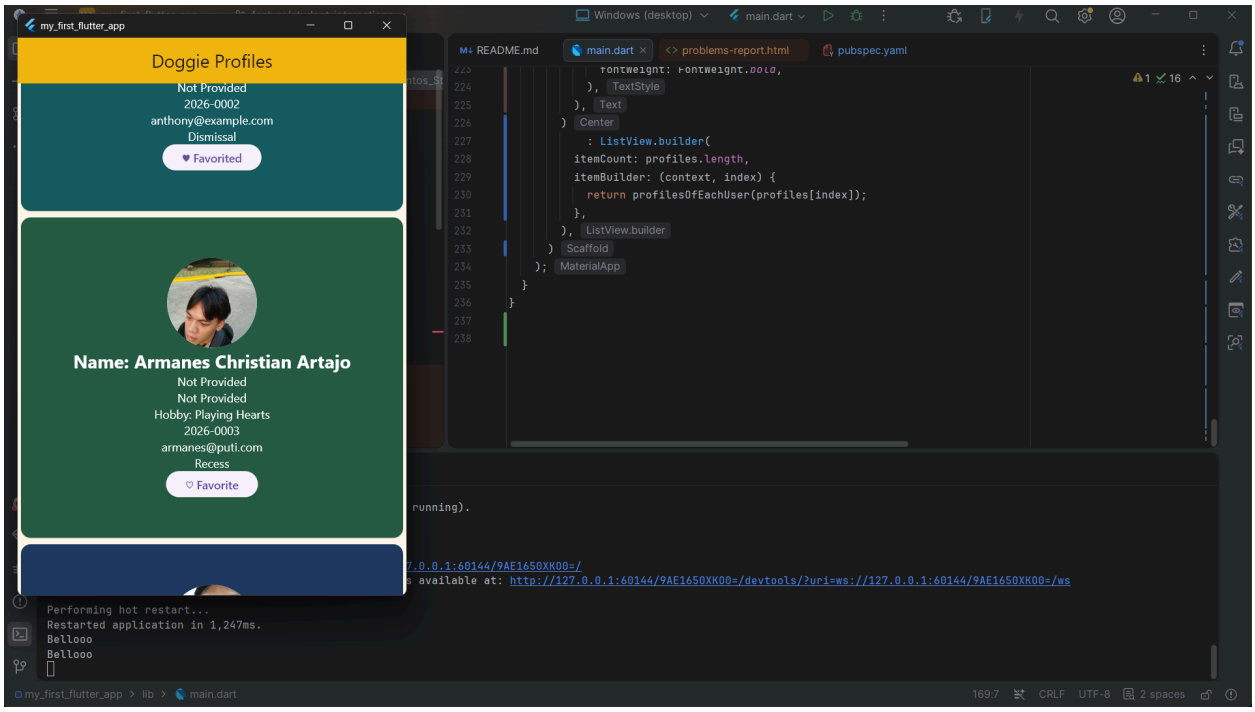
Screenshot:

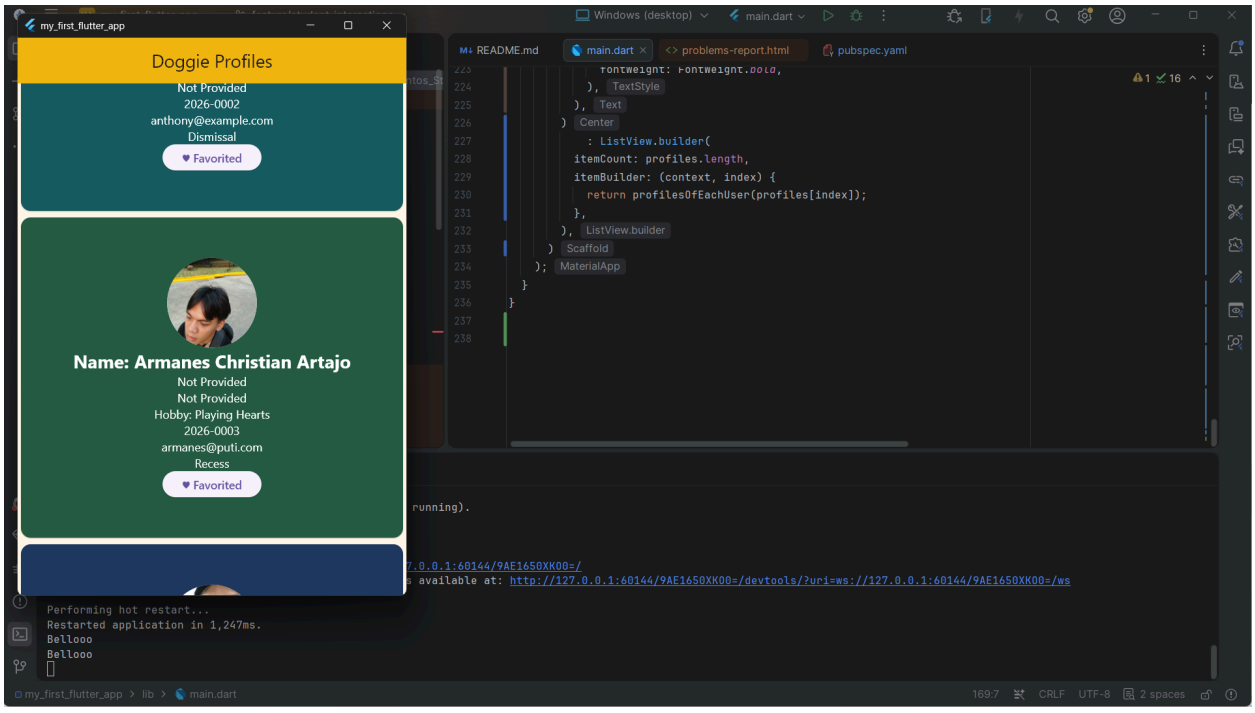
- `setState` in your code
- Before and after interaction

```dart
class _InteractiveFavButtonState extends State<InteractiveFavButton>
{
  bool isFavorite = false;

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () {
        setState(() {
          if (!isFavorite)
          {
            isFavorite = true;
          }
          else {
            isFavorite = !isFavorite;
          }
        });
      },

      child: Text(
        isFavorite
          ? "♥ Favorited"
          : "♡ Favorite",
      ), // Text
    ); // ElevatedButton
  }
}
```


my_first_flutter_app
Doggie Profiles
Not Provided
2026-0002
anthony@example.com
Dismissal
♥ Favorited
Name: Armanes Christian Artajo
Not Provided
Not Provided
Hobby: Playing Hearts
2026-0003
armanes@puti.com
Recess
♡ Favorite
Performing hot restart...
Restarted application in 1,247ms.
Bellooo
Bellooo
Windows (desktop)
main.dart
README.md
problems-report.html
pubspec.yaml
ListView.builder(
itemCount: profiles.length,
itemBuilder: (context, index) {
return profilesOfEachUser(profiles[index]);
},
),
Center
Scaffold
MaterialApp
http://127.0.0.1:60144/9AE1650XK00=/devtools/?uri=ws://127.0.0.1:60144/9AE1650XK00=/ws


my_first_flutter_app
Doggie Profiles
Not Provided
2026-0002
anthony@example.com
Dismissal
♥ Favorited
Name: Armanes Christian Artajo
Not Provided
Not Provided
Hobby: Playing Hearts
2026-0003
armanes@puti.com
Recess
♥ Favorited
Performing hot restart...
Restarted application in 1,247ms.
Bellooo
Bellooo
Windows (desktop)
main.dart
README.md
problems-report.html
pubspec.yaml
ListView.builder(
itemCount: profiles.length,
itemBuilder: (context, index) {
return profilesOfEachUser(profiles[index]);
},
),
Center
Scaffold
MaterialApp
http://127.0.0.1:60144/9AE1650XK00=/devtools/?uri=ws://127.0.0.1:60144/9AE1650XK00=/ws

🏴 Flag 4 — Favorite Students

Objective: Let administrators mark students as favorites.

Schools often need to highlight important records.

Add a favorite feature that changes a student's appearance when marked.

Possible visual changes include:

- Filled heart/star
- Different card color
- Favorite label

Each student's favorite status should work independently.

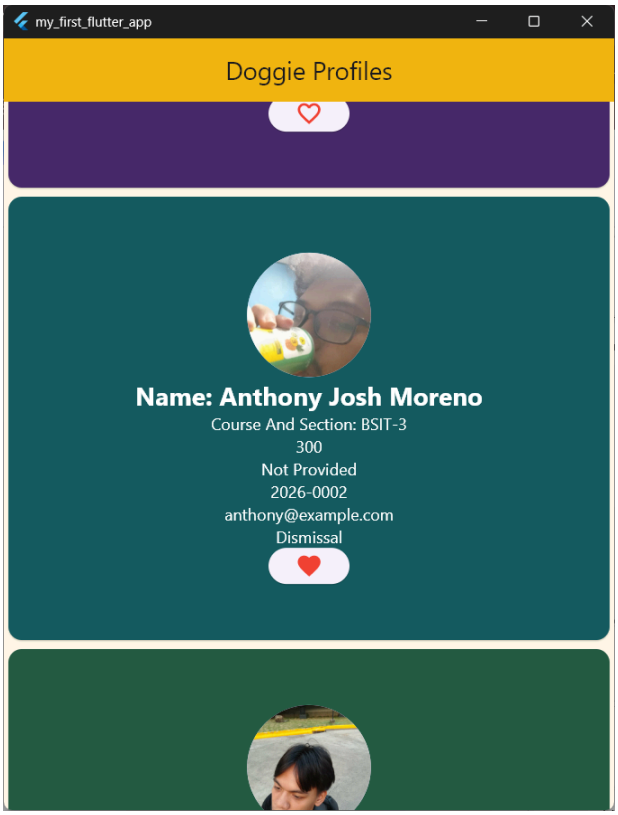
Test

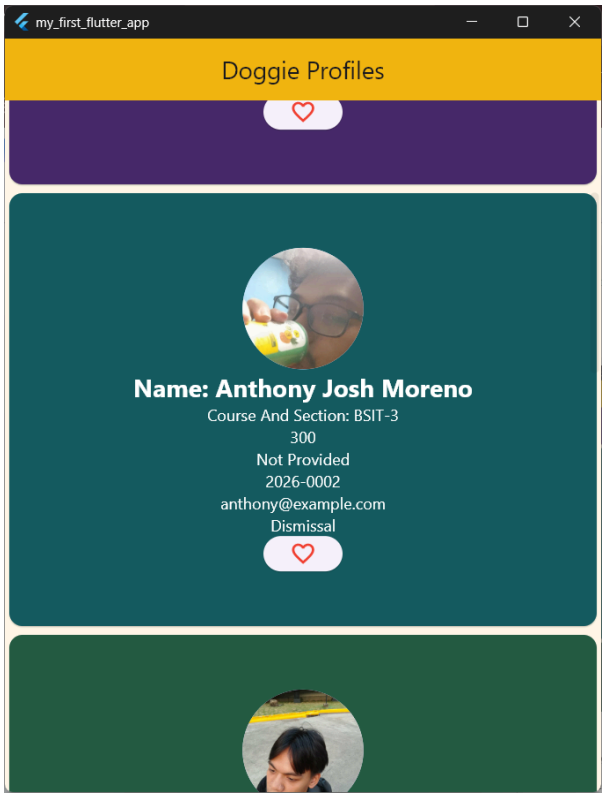
- Favorite one student.
- Other students should remain unchanged.
- Favorite another student.

Both should keep their own state.

Screenshot:

- Favorited student
- Non-favorited student
- Favorite icon changing





🏴 Flag 5 — Remove a Student

Objective: Delete a student from the directory.

Research how to remove an item from a Dart `List`.

Add a Delete action that removes a student from your list.

The UI should immediately update after deletion.

Test

Before:

- Student A
- Student B
- Student C

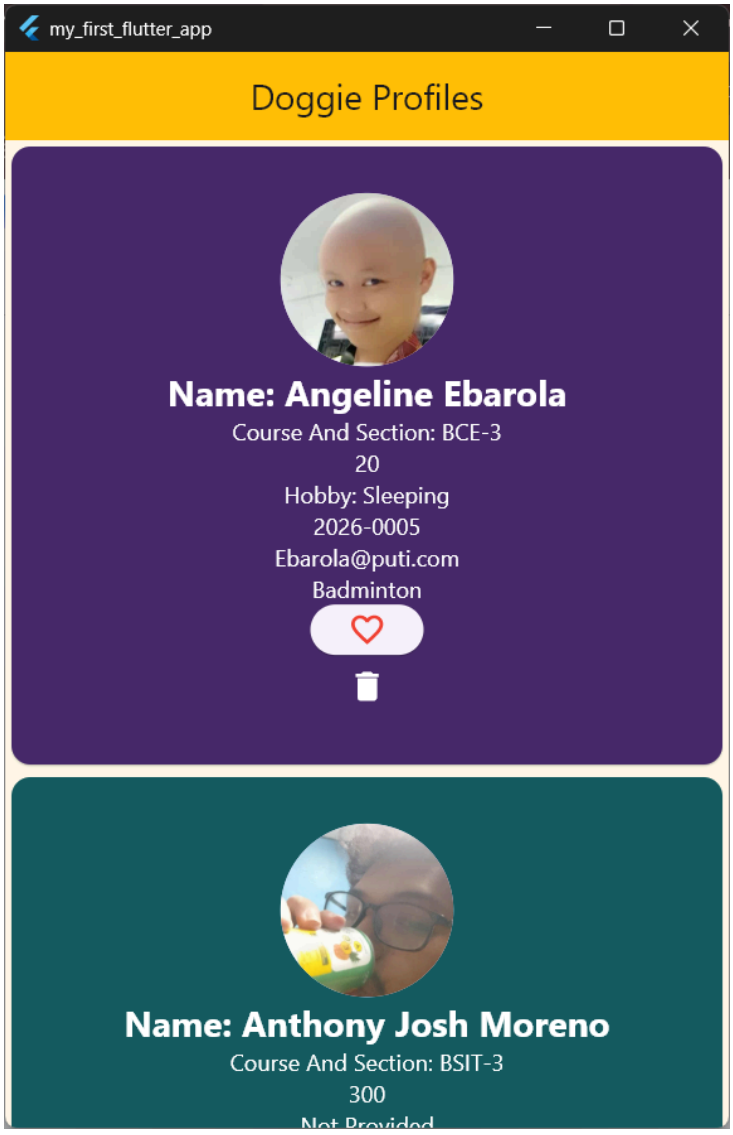
After deleting Student B:

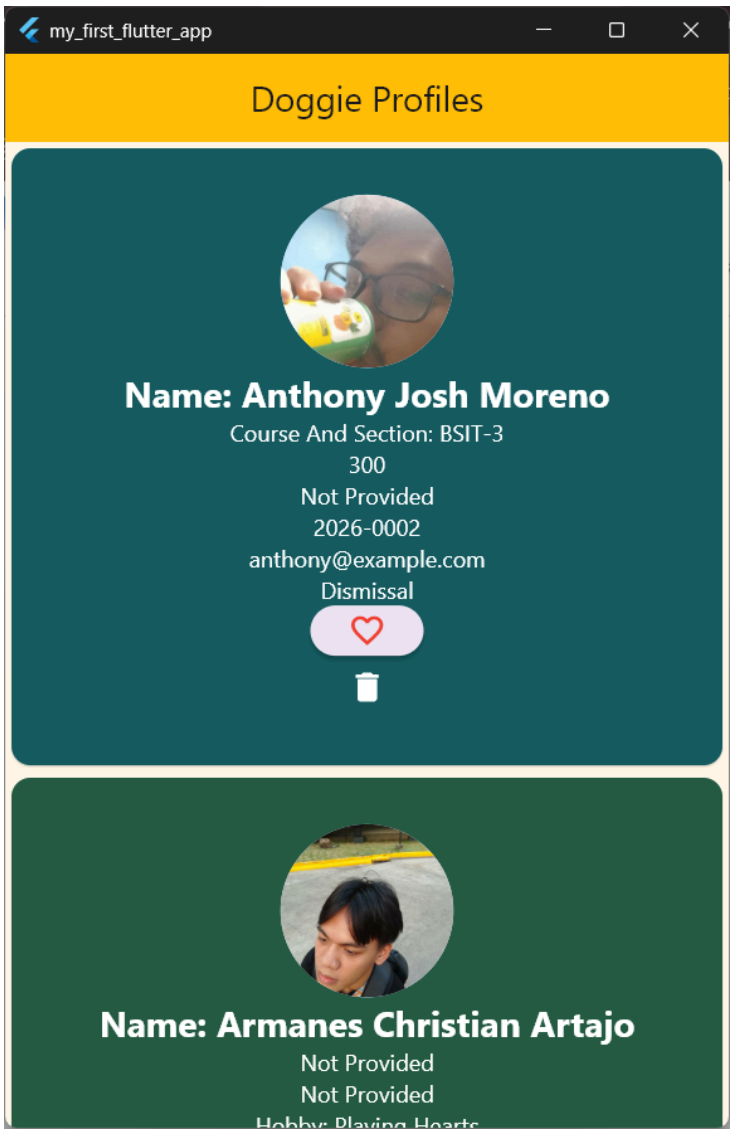
- Student A
- Student C

No empty spaces should remain.

Screenshot:

- Student before deletion
- Student after deletion





🏴 Flag 6 — Edit Trigger

Objective: Prepare your app for editing student information.

In the next CTF you'll build a full Edit Screen.

For now, create an Edit trigger.

Research simple ways Flutter can display temporary editing interfaces.

Examples include:

- `AlertDialog`
- `SnackBar`
- Bottom Sheet

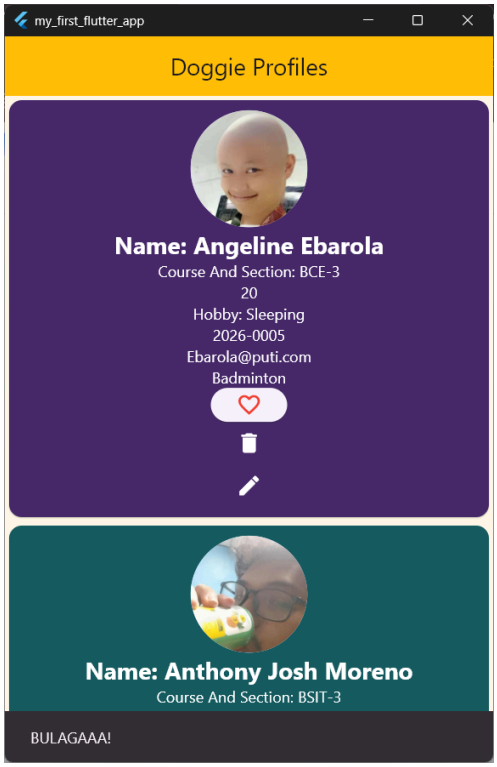
The goal is simply to launch an edit action, not build the complete editor yet.

Test

Pressing Edit should open your chosen interface.

Screenshot:

- Edit button
- Opened dialog, snackbar, or bottom sheet



🏴 Flag 7 — Multiple Actions, Independent State

Objective: Make every Student Card manage interactions correctly.

This is your final interaction challenge.

Verify that every student behaves independently.

Example scenario:

- Favorite Student A.
- Delete Student C.
- Open Edit for Student B.c

Your application should continue functioning correctly without affecting unrelated students.

Test Checklist

- Favorite works independently.
- Delete updates the list immediately.
- Edit still opens correctly.
- Remaining students keep their own states.

Screenshot:

- Multiple interactions working together
- Application still functioning normally

my_first_flutter_app

Doggie Profiles



Name: Angeline Ebarola
Course And Section: BCE-3
20
Hobby: Sleeping
2026-0005
Ebarola@puti.com
Badminton



Name: Anthony Josh Moreno
Course And Section: BSIT-3
300
Not Provided
2026-0002

my_first_flutter_app

Doggie Profiles



Name: Anthony Josh Moreno
Course And Section: BSIT-3
300
Not Provided
2026-0002
anthony@example.com
Dismissal



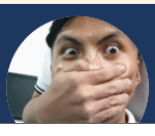
Name: Armanes Christian Artajo
Not Provided
Not Provided
Hobby: Playing Hearts

my_first_flutter_app

Doggie Profiles



Name: Armanes Christian Artajo
Not Provided
Not Provided
Hobby: Playing Hearts
2026-0003
armanes@puti.com
Recess

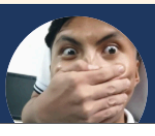


my_first_flutter_app

Doggie Profiles



Name: CHRISTIANNNN
Not Provided
Not Provided
Hobby: Playing Chess
2026-0003
armanes@puti.com
Sakading Padung Palambing



```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'my 2nd Flutter Design MWHEHEHHE',
      home: const Profiles(),
    ); // MaterialApp
  }
}

class Profile {
  String? name;
  String? courseAndSection;
  int? age;
  String? hobby;
  double? height;
  bool? isStudent;
  int color;
  String picture;
  String? studentId;
  String? email;
  String? favoriteSubject;
  Profile(
      {  this.name,  this.courseAndSection,  thi
        this.email,
        this.favoriteSubject,
      });
}

List<Profile> profiles = [
  Profile(
    name: 'Name: Carl Joshua Santos',
    hobby: 'Hobby: Sleeping',
    height: 5.8399,
    isStudent: true,
    color: 0xFF1E3A5F,
    picture: "assets/shh.jpg",
    studentId: '2026-0001',
    email: 'carl@puti.com',
    favoriteSubject: 'Astronomy',
  ),

  Profile(
    name: 'Name: Anthony Josh Moreno',
    courseAndSection: 'Course And Section: BSIT-3',
    age: 300,
    height: 5.4,
    isStudent: true,
    color: 0xFF155E63,
    picture: "assets/tonikss.jpg",
    studentId: '2026-0002',
    email: 'anthony@example.com',
    favoriteSubject: 'Dismissal',
  ),

  Profile(
    name: 'Name: Armanes Christian Artajo',
    hobby: 'Hobby: Playing Hearts',
    height: 5.8,
    isStudent: true,
    color: 0xFF245B45,
    picture: "assets/manes.jpg",
    studentId: '2026-0003',
    email: 'armanes@puti.com',
    favoriteSubject: 'Recess',
  ),
```

```
76    Profile(
77      name: 'Name: Krylle Jane Racaza',
78      courseAndSection: 'Course And Section: BSIT-3C',
79      hobby: 'Hobby: Sleeping',
80      height: 5.2,
81      isStudent: true,
82      color: 0xFF7A263A,
83      picture: "assets/karil.jpg",
84      studentId: '2026-0004',
85      email: 'racaza@puti.com',
86      favoriteSubject: 'Break',
87    ),
88
89    Profile(
90      name: 'Name: Angeline Ebarola',
91      courseAndSection: 'Course And Section: BCE-3',
92      age: 20,
93      hobby: 'Hobby: Sleeping',
94      height: 5.5,
95      isStudent: true,
96      color: 0xFF4A2C6D,
97      picture: "assets/priiHAHAHAHA.jpg",
98      studentId: '2026-0005',
99      email: 'Ebarola@puti.com',
100     favoriteSubject: 'Badminton',
101   ),
102
103   Profile(
104     name: 'Name: Keith Lopez',
105     courseAndSection: 'Course And Section: BSIT-3',
106     age: 20,
107     hobby: 'Hobby: Scatter',
108     height: 5.11,
109     isStudent: true,
110     color: 0xFFA52A2A,
111     picture: "assets/Keith.jpg",
112     studentId: '2026-0006',
```

```
118 class InteractiveFavButton extends StatefulWidget{
119   const InteractiveFavButton({super.key});
120
121   @override
122   State<InteractiveFavButton> createState() => _InteractiveFavButtonState();
123 }
124
125 class _InteractiveFavButtonState extends State<InteractiveFavButton>
126 {
127   bool isFavorite = false;
128
129   @override
130   Widget build(BuildContext context) {
131     return ElevatedButton(
132       onPressed: () {
133         setState(() {
134           isFavorite = !isFavorite;
135         });
136       },
137       child: isFavorite ?
138       const Icon(
139         Icons.favorite,
140         color: Colors.red,
141         size: 24.0,
142         semanticLabel: 'Make it Reddd',
143       ) : // Icon
144       const Icon(
145         Icons.favorite_border,
146         color: Colors.red,
147         size: 24.0,
148         semanticLabel: 'Not red',
149       ) // Icon
150     ); // ElevatedButton
151   }
152 }
```

```dart
class Profiles extends StatefulWidget {
  const Profiles({super.key});

  @override
  State<Profiles> createState() => _ProfileState();
}

class _ProfileState extends State<Profiles>
{
  bool isDeleted = false;

  Widget profilesOfEachUser(Profile profile)
  {
    return SizedBox(
      width: 350,
      height: 400,
      child: GestureDetector(
        onTap: ()
        {
          print("Bellooo");    Don't invoke 'print' in production code.↵Try using a logging framework.
        },
        child: Card(
          color: Color(profile.color),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            crossAxisAlignment: CrossAxisAlignment.center,
            children: [
              CircleAvatar(
                radius: 55,
                backgroundImage: AssetImage(profile.picture),
              ), // CircleAvatar
              Text(profile.name ?? "Name Unknown",
                style: const TextStyle(
                  color : Colors.white,
                  fontSize: 22,
                  fontWeight: FontWeight.bold) // TextStyle
              ), // Text
              Text(profile.courseAndSection ?? "Not Provided",style: TextStyle(color: Colors.white)),
              Text(profile.age?.toString() ?? "Not Provided",style: TextStyle(color: Colors.white)),
              Text(profile.hobby ?? "Not Provided",style: TextStyle(color: Colors.white)),
              Text(profile.studentId ?? "Not Provided",style: TextStyle(color: Colors.white)),
              Text(profile.email ?? "Not Provided",style: TextStyle(color: Colors.white)),
              Text(profile.favoriteSubject ?? "Not Provided",style: TextStyle(color: Colors.white)),
              InteractiveFavButton(),
              IconButton(
                icon: const Icon(Icons.delete, color: Colors.white),
                onPressed:  ()
                {
                  setState(() {
                    profiles.remove(profile);
                  });
                }
              ), // IconButton
              IconButton(
                icon: const Icon(
                  Icons.edit,
                  color: Colors.white,
                ), // Icon
                onPressed: () {
                  TextEditingController nameController =TextEditingController(text: profile.name);
                  TextEditingController hobbyController = TextEditingController(text: profile.hobby);
                  TextEditingController favsubController = TextEditingController(text: profile.favoriteSubject);
                  showDialog(
                    context: context,
                    builder: (context) {
                      return AlertDialog(
                        title: const Text('Edit Student'),
                        content: Column(
                          mainAxisSize: MainAxisSize.min,
                          children: [
                            TextField(
                              controller: nameController,
                              decoration: const InputDecoration(
```

```
                    children: [
                      TextField(
                        controller: nameController,
                        decoration: const InputDecoration(
                          labelText: 'Name',
                        ), // InputDecoration
                      ), // TextField
                      TextField(
                        controller: hobbyController,
                        decoration: const InputDecoration(
                          labelText: 'Hobby',
                        ), // InputDecoration
                      ), // TextField
                      TextField(
                        controller: favsubController,
                        decoration: const InputDecoration(
                          labelText: 'Favorite Subject',
                        ), // InputDecoration
                      ) // TextField
                    ]
                  ), // Column
                  actions: [
                    TextButton(
                      onPressed: () {
                        Navigator.pop(context);
                      },
                      child: const Text('Cancel'),
                    ), // TextButton
                    TextButton(
                      onPressed: () {
                        setState(() {
                          profile.name = nameController.text;
                          profile.hobby = hobbyController.text;
                          profile.favoriteSubject = favsubController.text;
                        });
                        Navigator.pop(context);
                      child: const Text('Cancel'),
                    ), // TextButton
                    TextButton(
                      onPressed: () {
                        setState(() {
                          profile.name = nameController.text;
                          profile.hobby = hobbyController.text;
                          profile.favoriteSubject = favsubController.text;
                        });
                        Navigator.pop(context);
                      },
                      child: const Text('Save'),
                    ), // TextButton
                  ],
                ); // AlertDialog
              },
            );
          },
        ), // IconButton

      ]
    ), // Column
  ), // Card
) // GestureDetector
); // SizedBox
}

@override
Widget build(BuildContext context) {
  profiles.sort((a,b) => (a.name ?? "").compareTo(b.name ?? ""));
  return Scaffold(
      backgroundColor: const Color(0xFFFFF8E7),
      appBar: AppBar(
        title: const Text("Doggie Profiles"),
        backgroundColor: Colors.amber,
        centerTitle: true,
```

```dart
  @override
  Widget build(BuildContext context) {
    profiles.sort((a,b) => (a.name ?? "").compareTo(b.name ?? ""));
    return Scaffold(
        backgroundColor: const Color(0xFFFFF8E7),
        appBar: AppBar(
          title: const Text("Doggie Profiles"),
          backgroundColor: Colors.amber,
          centerTitle: true,
        ), // AppBar
        body: profiles.isEmpty
            ? const Center(
          child: Text(
            "No students found.",
            style: TextStyle(
              fontSize: 24,
              fontWeight: FontWeight.bold,
            ), // TextStyle
          ), // Text
        ) // Center
            : ListView.builder(
          itemCount: profiles.length,
          itemBuilder: (context, index) {
            return profilesOfEachUser(profiles[index]);
          },
        ), // ListView.builder
    ); // Scaffold
  }
}
```

🏴 Flag 8 — Save Your Progress

Objective: Commit and push your completed Interactive Student Directory.

Use a proper Git commit message.

Branch:

`feature/student-interactions`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

